

CLI reference

calepin

Preprocess Typst documents with executable code chunks

Usage: calepin <COMMAND>

Commands:

new	Create a notebook file, website scaffold, or ejected theme
health	Check Calepin's local runtime environment and local links
compile	Preprocess, then invoke typst compile
watch	Watch and preprocess, optionally delegating rendering to typst watch
serve	Serve static files locally
update	Update Calepin using the official installer updater
clean	Remove <code>`.calepin`</code> directories and generated artifacts
help	Print this message or the help of the given subcommand(s)

Options:

-v, --version	Print version
-h, --help	Print help

calepin new

Preprocess Typst documents with executable code chunks

Usage: calepin <COMMAND>

Commands:

new	Create a notebook file, website scaffold, or ejected theme
health	Check Calepin's local runtime environment and local links
compile	Preprocess, then invoke typst compile
watch	Watch and preprocess, optionally delegating rendering to typst watch
serve	Serve static files locally
update	Update Calepin using the official installer updater
clean	Remove <code>`.calepin`</code> directories and generated artifacts
help	Print this message or the help of the given subcommand(s)

Options:

-v, --version	Print version
-h, --help	Print help

calepin health

Create a notebook file, website scaffold, or ejected theme

Usage: calepin new [OPTIONS] <PATH|website|theme> [DIR]

Arguments:

<PATH website theme>	What to create: a .typ notebook path, <code>`website`</code> , or <code>`theme`</code>
[DIR]	Destination directory when creating a website scaffold or ejected theme

Options:

--theme <THEME>	Built-in theme to use when creating a website scaffold or
-----------------	---

```
ejected theme [possible values: calepin, academic]
-f, --force      Overwrite the file if it already exists
-h, --help      Print help
```

Examples:

```
calepin new paper.typ
calepin new website
calepin new website --theme academic
calepin new theme
calepin new theme --theme academic
calepin new theme themes/my-theme
```

calepin compile

Create a notebook file, website scaffold, or ejected theme

Usage: calepin new [OPTIONS] <PATH|website|theme> [DIR]

Arguments:

```
<PATH|website|theme>  What to create: a .typ notebook path, `website`, or `theme`
[DIR]                 Destination directory when creating a website scaffold or
ejected theme
```

Options:

```
--theme <THEME>      Built-in theme to use when creating a website scaffold or
ejected theme [possible values: calepin, academic]
-f, --force          Overwrite the file if it already exists
-h, --help          Print help
```

Examples:

```
calepin new paper.typ
calepin new website
calepin new website --theme academic
calepin new theme
calepin new theme --theme academic
calepin new theme themes/my-theme
```

calepin watch

Check Calepin's local runtime environment and local links

Usage: calepin health [OPTIONS]

Options:

```
--config <CONFIG>      Path to project config TOML
-d, --depth <DEPTH>    Maximum recursion depth when searching for links
--json                 Print machine-readable JSON
--strict               Exit with an error when warnings are present
--check-external-links Also check external links
-h, --help             Print help
```

calepin serve

Check Calepin's local runtime environment and local links

Usage: calepin health [OPTIONS]

Options:

<code>--config <CONFIG></code>	Path to project config TOML
<code>-d, --depth <DEPTH></code>	Maximum recursion depth when searching for links
<code>--json</code>	Print machine-readable JSON
<code>--strict</code>	Exit with an error when warnings are present
<code>--check-external-links</code>	Also check external links
<code>-h, --help</code>	Print help

calepin update

Preprocess, then invoke typst compile

Usage: calepin compile [OPTIONS] <INPUT> [OUTPUT] [TYPST_ARGS]...

Arguments:

`<INPUT>`
Input .typ file, or a website source directory containing calepin.toml

`[OUTPUT]`
Output file path, or website output directory when INPUT is a directory

`[TYPST_ARGS]...`
Arguments forwarded to typst compile after `--`

Options:

<code>--format <FORMAT></code>	Output format, including source-code script extraction
	[possible values: pdf, png, svg, html, script]
<code>--minify</code>	Minify HTML output after theming and asset processing
<code>--config <CONFIG></code>	Path to project config TOML
<code>-q, --quiet</code>	Quiet mode
<code>--timeout <TIMEOUT></code>	Per-chunk timeout in seconds
<code>--set <KEY=VALUE></code>	Override a Calepin config value as `key=value` (repeatable).
	Uses dotted paths for nested config, such as `theme=./theme`, `vars.region=CA`, or `toc.enabled=false`.
<code>-h, --help</code>	Print help (see a summary with '-h')

calepin clean

Preprocess, then invoke typst compile

Usage: calepin compile [OPTIONS] <INPUT> [OUTPUT] [TYPST_ARGS]...

Arguments:

<INPUT>

Input .typ file, or a website source directory containing calepin.toml

[OUTPUT]

Output file path, or website output directory when INPUT is a directory

[TYPST_ARGS]...

Arguments forwarded to typst compile after `--`

Options:

--format <FORMAT>

Output format, including source-code script extraction

[possible values: pdf, png, svg, html, script]

--minify

Minify HTML output after theming and asset processing

--config <CONFIG>

Path to project config TOML

-q, --quiet

Quiet mode

--timeout <TIMEOUT>

Per-chunk timeout in seconds

--set <KEY=VALUE>

Override a Calepin config value as `key=value` (repeatable).

Uses dotted paths for nested config, such as `theme=./theme`,
`vars.region=CA`, or `toc.enabled=false`.

-h, --help

Print help (see a summary with '-h')